Spring JDBC: From Raw JDBC to `JdbcTemplate`

Spring JDBC reduces the repetitive infrastructure code required by raw JDBC while keeping SQL under the developer's control.

With Spring JDBC, we still write:

- SQL queries
- Query parameters
- Row-to-object mapping
- Application-specific result handling

Spring takes care of:

- Acquiring database connections
- Creating and executing statements
- Closing JDBC resources
- Translating SQL exceptions

---

1. The Problem with Raw JDBC

Consider a simple requirement: insert a student into the database.

```
INSERT INTO students(name, email, age)
VALUES (?, ?, ?);
```

Using raw JDBC, the repository method may look like this:

```
public boolean save(Student student) {

    String sql = """
            INSERT INTO students(name, email, age)
```

```
            VALUES (?, ?, ?)
            """;

    try (
        Connection connection =
                DriverManager.getConnection(URL, USERNAME, PA
SSWORD);

        PreparedStatement preparedStatement =
                connection.prepareStatement(sql)
    ) {
        preparedStatement.setString(1, student.getName());
        preparedStatement.setString(2, student.getEmail());
        preparedStatement.setInt(3, student.getAge());

        int rowsAffected = preparedStatement.executeUpdate();

        return rowsAffected == 1;
    }
    catch (SQLException exception) {
        exception.printStackTrace();
        return false;
    }
}
```

The actual database-specific work is limited to:

```
String sql = "...";

preparedStatement.setString(1, student.getName());
preparedStatement.setString(2, student.getEmail());
preparedStatement.setInt(3, student.getAge());

preparedStatement.executeUpdate();
```

The remaining code manages JDBC infrastructure:

- Opening a connection
- Creating a `PreparedStatement`
- Closing the statement
- Closing the connection
- Catching `SQLException`
- Repeating the same workflow in every repository method

The same structure appears in insert, update, delete and select operations.

> Spring JDBC removes this repeated workflow without removing our control over SQL.

---

2. The Template Idea Behind Spring JDBC

Most JDBC operations follow the same general algorithm:

```
1. Obtain a database connection
2. Create a statement
3. Bind parameter values
4. Execute the statement
5. Process the result
6. Handle SQL exceptions
7. Close JDBC resources
```

Some parts remain almost identical for every operation:

```
Obtain connection
Create statement
Execute statement
Handle exceptions
Close resources
```

Other parts change for each query:

```
SQL statement
Parameter values
```

```
Result mapping
```

Spring separates these responsibilities.

Spring handles	Developer handles
Connection management	SQL query
Statement execution workflow	SQL parameter values
Resource cleanup	Row-to-object mapping
Exception translation	Business interpretation of the result

This follows the **Template Method idea**: Spring provides the fixed JDBC workflow, while the developer supplies the operation-specific parts.

The central class used for this purpose is:

```
JdbcTemplate
```

Spring JDBC still uses standard JDBC objects internally:

```
Connection
PreparedStatement
ResultSet
SQLException
```

The difference is that Spring manages their lifecycle for us.

---

3. Understanding `DataSource`

Before using `JdbcTemplate`, we need to understand where database connections come from.

That responsibility belongs to `DataSource`.

3.1 What Is a `DataSource`?

`DataSource` is an interface provided by Java:

```
javax.sql.DataSource
```

Its main responsibility is to provide database connections.

Conceptually, its important method is:

```
Connection getConnection();
```

A `DataSource` is therefore a **connection provider**.

```
DataSource
    ↓
Provides Connection objects
```

A `DataSource` is not itself a database connection.

```
Connection connection = dataSource.getConnection();
```

3.2 `DriverManager` vs `DataSource`

With raw JDBC, a connection is commonly created using `DriverManager`:

```
Connection connection =
        DriverManager.getConnection(
                URL,
                USERNAME,
                PASSWORD
        );
```

This makes the repository responsible for knowing:

- The database URL
- The username
- The password

- How connections are created

A repository should mainly be concerned with database operations related to its domain, such as student queries.

With `DataSource`, a connection provider can be injected:

```
public class StudentRepository {

    private final DataSource dataSource;

    public StudentRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }
}
```

The repository can then request a connection without knowing how it was created:

```
Connection connection = dataSource.getConnection();
```

`DriverManager`	`DataSource`
Static connection-acquisition mechanism	Injectable connection-provider abstraction
Repository knows connection details	Connection configuration remains outside the repository
Common in small raw JDBC programs	Preferred in managed applications
Does not itself represent pooling	Can be implemented by a connection pool

4. Connection Pooling

A connection pool maintains a controlled collection of database connections that can be reused by the application.

> Instead of creating a new physical connection for every query, the application borrows an existing connection and returns it after use.

4.1 Connection Creation Without a Pool

Creating a database connection may involve several steps:

```
Java application
      ↓
Network connection established
      ↓
Database server contacted
      ↓
Credentials verified
      ↓
Database session created
      ↓
Connection returned to the application
```

Without pooling, every operation may follow this lifecycle:

```
Create connection
      ↓
Execute SQL
      ↓
Physically close connection
```

For a small console application, this may be acceptable.

For a web application handling many concurrent requests, repeatedly creating physical connections becomes expensive.

```
Request 1 → Create connection → Query → Close
Request 2 → Create connection → Query → Close
Request 3 → Create connection → Query → Close
```

4.2 How a Connection Pool Works

A pool creates and manages a limited number of database connections.

```
Connection Pool
├── Connection 1
├── Connection 2
```

```
├── Connection 3
├── Connection 4
└── Connection 5
```

When repository code requests a connection:

```
Connection connection = dataSource.getConnection();
```

the pool returns an available connection.

When the code calls:

```
connection.close();
```

the connection is generally not physically destroyed. The pool intercepts the call and returns the connection for reuse.

```
Borrow connection
      ↓
Execute database operation
      ↓
Call close()
      ↓
Connection returned to pool
```

4.3 What Happens When Every Connection Is Busy?

Assume the maximum pool size is three:

```
Connection A → In use
Connection B → In use
Connection C → In use
```

A fourth request asks for a connection:

```
dataSource.getConnection();
```

Because no connection is currently available, the request waits.

```
Request 4
    ↓
Wait for an available connection
```

When another request finishes, its connection returns to the pool and becomes available.

If no connection becomes available within the configured timeout, the pool throws an exception.

This protects the database from an unlimited number of simultaneous connections.

4.4 Why Connections Must Be Limited

Each database connection consumes resources such as:

- Memory
- Socket resources
- Session state
- Transaction-related resources
- Database processing capacity

Allowing every incoming request to create its own connection can overwhelm the database.

A connection pool therefore improves:

- Performance through connection reuse
- Stability through controlled concurrency
- Resource management on both the application and database sides

---

5. `DataSource` Does Not Always Mean Connection Pooling

`DataSource` is only an interface.

Different implementations can behave differently:

- One implementation may create a new physical connection for every call.
- Another implementation may return an existing connection from a pool.

Therefore:

```
DataSource ≠ Connection Pool
```

Instead:

```
A connection pool can implement DataSource
```

For example:

```
HikariDataSource
```

is a `DataSource` implementation backed by the HikariCP connection pool.

5.1 Why `DataSource` Is a Better Abstraction

Repository code depends on the `DataSource` interface:

```
Connection connection = dataSource.getConnection();
```

The implementation can change without changing repository logic.

```
Development
DataSource
    ↓
Simple connection provider
```

```
Production
DataSource
```

```
    ↓
HikariCP connection pool
```

The repository depends on the abstraction:

```
DataSource
```

rather than a specific implementation:

```
HikariDataSource
```

This is a practical example of dependency inversion.

---

6. HikariCP

HikariCP is a JDBC connection-pool implementation.

Spring Boot commonly uses HikariCP when it is available on the classpath. It is included by default with starters such as:

```
spring-boot-starter-jdbc
spring-boot-starter-data-jpa
```

HikariCP manages concerns such as:

- Maximum pool size
- Minimum idle connections
- Connection timeout
- Idle timeout
- Maximum connection lifetime
- Borrowing and returning connections
- Connection validation

Common configuration properties include:

```
maximumPoolSize
minimumIdle
connectionTimeout
idleTimeout
maxLifetime
```

---

7. Creating a Spring JDBC Project

7.1 Required Dependencies

Add the Spring JDBC starter:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

Add the MySQL JDBC driver:

```xml
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

For a REST application, also include:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

The JDBC starter provides Spring JDBC infrastructure.

The MySQL dependency provides the database-specific JDBC driver.

7.2 Database Configuration

Configure the database in `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
spring.datasource.password=your_password
```

Spring Boot reads datasource settings from:

```
spring.datasource.*
```

Spring Boot can normally determine the JDBC driver from the connection URL, so explicitly configuring the driver class is usually unnecessary.

When required, it can be set using:

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

7.3 HikariCP Configuration

Pool-specific settings can be placed under:

```
spring.datasource.hikari.*
```

Example:

```
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
```

Meaning of the Main Properties

Property	Purpose
`maximum-pool-size`	Maximum number of connections managed by the pool
`minimum-idle`	Minimum number of idle connections the pool attempts to maintain
`connection-timeout`	Maximum waiting time for a connection before failure
`idle-timeout`	Time an idle connection may remain in the pool before removal
`max-lifetime`	Maximum lifetime of a pooled connection

> A larger pool does not automatically improve performance.

Pool size should be selected using:

- Expected concurrency
- Average query duration
- Database capacity
- Number of application instances
- Load-test results

Every connection consumes resources in both the application and the database.

---

8. How Spring Boot Creates the `DataSource` Bean

Spring Boot performs a process conceptually similar to the following:

```
1. Detect Spring JDBC on the classpath
2. Detect the JDBC driver
3. Read spring.datasource.* properties
4. Detect an available connection-pool implementation
5. Prefer HikariCP when available
6. Create and configure a DataSource
7. Register the DataSource as a Spring bean
```

Conceptually, Spring Boot creates something similar to:

```
@Bean
public DataSource dataSource() {

    HikariDataSource dataSource = new HikariDataSource();

    dataSource.setJdbcUrl(
            "jdbc:mysql://localhost:3306/student_db"
    );

    dataSource.setUsername("root");
    dataSource.setPassword("password");

    return dataSource;
}
```

This bean normally does not need to be created manually because Spring Boot auto-configuration creates it.

When an application defines its own `DataSource` bean, Spring Boot generally backs away from its default datasource auto-configuration.

8.1 Verifying the `DataSource` Implementation

The configured implementation can be verified by injecting the bean:

```
@Component
public class DataSourceChecker {

    private final DataSource dataSource;

    public DataSourceChecker(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @PostConstruct
    public void checkDataSource() {
```

```
        System.out.println(dataSource.getClass());
    }
}
```

A typical output is:

```
class com.zaxxer.hikari.HikariDataSource
```

---

9. Introducing `JdbcTemplate`

`JdbcTemplate` is Spring's main abstraction for performing JDBC operations.

```
org.springframework.jdbc.core.JdbcTemplate
```

It provides operations for:

- Insert
- Select
- Update
- Delete
- Batch operations
- Stored procedures
- Generated-key retrieval

The most commonly used methods are:

```
update()
query()
queryForObject()
```

9.1 Relationship Between `JdbcTemplate` and `DataSource`

`JdbcTemplate` does not create database connections independently. It requires a `DataSource`.

```
JdbcTemplate
    ↓ requests a connection from
DataSource
    ↓ provides
Connection
```

Conceptually, the bean can be created as follows:

```
@Bean
public JdbcTemplate jdbcTemplate(DataSource dataSource) {
    return new JdbcTemplate(dataSource);
}
```

Spring Boot normally configures this bean automatically when Spring JDBC and a valid `DataSource` are available.

The configured template can then be injected:

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

There is usually no need to create an unconfigured instance manually:

```
new JdbcTemplate();
```

9.2 Responsibilities Handled by `JdbcTemplate`

For an update operation:

```
jdbcTemplate.update(sql, parameters);
```

Spring conceptually performs:

```
1. Ask DataSource for a connection
2. Create a PreparedStatement
3. Bind parameter values
4. Execute the statement
5. Read the affected-row count
6. Close the statement
7. Release the connection
8. Translate SQLException when necessary
```

For a select operation:

```
1. Obtain a connection
2. Create a PreparedStatement
3. Bind parameter values
4. Execute the query
5. Obtain the ResultSet
6. Iterate through the rows
7. Call the RowMapper for each row
8. Collect the mapped objects
9. Close the ResultSet
10. Close the statement
11. Release the connection
12. Translate SQL exceptions
```

10. Project Structure

A simple project structure can be:

```
src/main/java
└── in/coderarmy/springjdbc
    ├── SpringJdbcApplication.java
```

```
    ├── controller
    │   └── StudentController.java
    ├── service
    │   └── StudentService.java
    ├── repository
    │   ├── StudentRepository.java
    │   └── StudentRowMapper.java
    └── model
        └── Student.java
```

10.1 Database and Table

```sql
CREATE DATABASE student_db;

USE student_db;

CREATE TABLE students (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    age INT NOT NULL
);
```

10.2 `Student` Model

```java
package in.coderarmy.springjdbc.model;

public class Student {

    private Long id;
    private String name;
    private String email;
    private Integer age;

    // No-argument constructor, parameterized constructor,
```

```
    // getters and setters
}
```


11. Create Operation with `JdbcTemplate`

The `update()` method is used for SQL statements that change database state:

```
INSERT
UPDATE
DELETE
```

11.1 Saving a Student

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public boolean save(Student student) {

        String sql = """
                INSERT INTO students(name, email, age)
                VALUES (?, ?, ?)
                """;

        int rowsAffected = jdbcTemplate.update(
                sql,
                student.getName(),
                student.getEmail(),
                student.getAge()
```

```
        );

        return rowsAffected == 1;
    }
}
```

The values are bound according to their position:

```
First ?  → student.getName()
Second ? → student.getEmail()
Third ?  → student.getAge()
```

Parameter order must match placeholder order.

The following call is incorrect:

```
jdbcTemplate.update(
        sql,
        student.getAge(),
        student.getName(),
        student.getEmail()
);
```

The first placeholder represents `name`, not `age`.

Spring uses prepared-statement-based execution internally when parameter values are supplied to the appropriate `update()` overload.

11.2 Affected-Row Count

`update()` returns an `int` representing the number of affected rows.

```
int rowsAffected = jdbcTemplate.update(...);
```

For an insert that should create one row:

```
return rowsAffected == 1;
```

Spring removes the need for:

- Explicit `Connection` handling
- Explicit `PreparedStatement` creation
- Individual parameter-setter calls
- `try-with-resources`
- Manual resource cleanup
- Repeated `SQLException` catch blocks

Spring does not remove responsibility for:

- Writing correct SQL
- Supplying parameters in the correct order
- Interpreting the affected-row count

---

12. Read Operations with `JdbcTemplate`

A write operation usually returns an affected-row count.

A select operation returns rows and columns that must be converted into Java objects.

Spring uses `RowMapper` for this conversion.

12.1 What Is `RowMapper`?

`RowMapper<T>` is an interface whose responsibility is:

```
Convert one ResultSet row into one Java object
```

Its central method is:

```
T mapRow(
        ResultSet resultSet,
        int rowNumber
) throws SQLException;
```

For a student query:

```
One database row
        ↓
StudentRowMapper
        ↓
One Student object
```

If a query returns ten rows, Spring calls the mapper ten times.

```
Row 1  → Student 1
Row 2  → Student 2
Row 3  → Student 3
...
Row 10 → Student 10
```

Spring handles result-set iteration. The mapper handles conversion of one row.

12.2 Creating `StudentRowMapper`

```
package in.coderarmy.springjdbc.repository;

import in.coderarmy.springjdbc.model.Student;
import org.springframework.jdbc.core.RowMapper;

import java.sql.ResultSet;
import java.sql.SQLException;

public class StudentRowMapper implements RowMapper<Student> {

    @Override
```

```
    public Student mapRow(
            ResultSet resultSet,
            int rowNumber
    ) throws SQLException {

        Student student = new Student();

        student.setId(
                resultSet.getLong("id")
        );

        student.setName(
                resultSet.getString("name")
        );

        student.setEmail(
                resultSet.getString("email")
        );

        student.setAge(
                resultSet.getInt("age")
        );

        return student;
    }
}
```

12.3 Fetching All Students with `query()`

Use `query()` when zero, one or multiple rows may be returned.

```
private final RowMapper<Student> studentRowMapper =
        new StudentRowMapper();

public List<Student> findAll() {
```

```
    String sql = """
            SELECT id, name, email, age
            FROM students
            """;

    return jdbcTemplate.query(
            sql,
            studentRowMapper
    );
}
```

The return type is:

```
List<Student>
```

Possible results:

```
No rows:
[]

Three rows:
[Student1, Student2, Student3]
```

`query()` normally returns an empty list when no rows match.

This makes it suitable for collection-based queries.

Internally, the workflow is conceptually similar to:

```
while (resultSet.next()) {
    Student student =
            rowMapper.mapRow(resultSet, rowNumber);

    results.add(student);
}
```

12.4 Reusing a Mapper

A stateless mapper can be reused instead of creating a new instance for every query.

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    private final RowMapper<Student> studentRowMapper =
            new StudentRowMapper();

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }
}
```

The mapper only reads the current `ResultSet` row and creates a new object, so it does not need mutable shared state.

12.5 Lambda-Based `RowMapper`

Because `RowMapper` is a functional interface, it can also be implemented with a lambda:

```
private final RowMapper<Student> studentRowMapper =
        (resultSet, rowNumber) ->
                new Student(
                        resultSet.getLong("id"),
                        resultSet.getString("name"),
                        resultSet.getString("email"),
                        resultSet.getInt("age")
                );
```

A separate mapper class makes the responsibility easier to understand during teaching.

A lambda is convenient for short and simple mappings.

---

13. Fetching One Student with `queryForObject()`

Use `queryForObject()` when the query is expected to return exactly one row.

```
public Student findById(Long id) {

    String sql = """
            SELECT id, name, email, age
            FROM students
            WHERE id = ?
            """;

    return jdbcTemplate.queryForObject(
            sql,
            studentRowMapper,
            id
    );
}
```

`queryForObject()` communicates the following expectation:

> This query must return exactly one result.

It does not mean that Spring will return `null` when no row exists.

13.1 Behaviour of `queryForObject()`

Exactly One Row

```
Returns the mapped Student object
```

Zero Rows

```
Throws EmptyResultDataAccessException
```

More Than One Row

```
Throws IncorrectResultSizeDataAccessException
```

For a primary-key query such as:

```
WHERE id = ?
```

more than one row should be impossible because the primary key is unique.

The common missing-data case is zero rows.

13.2 Returning `Optional<Student>`

A missing row can be represented using `Optional.empty()`:

```
public Optional<Student> findById(Long id) {

    String sql = """
            SELECT id, name, email, age
            FROM students
            WHERE id = ?
            """;

    try {
        Student student =
                jdbcTemplate.queryForObject(
                        sql,
```

```
                        studentRowMapper,
                        id
                );

            return Optional.of(student);
        }
        catch (EmptyResultDataAccessException exception) {
            return Optional.empty();
        }
    }
```

Only `EmptyResultDataAccessException` is converted into an empty optional.

Do not catch every `DataAccessException` and return `Optional.empty()`.

These situations are different:

```
Student does not exist
```

```
Database server is unavailable
```

Only the first case represents an absent student.

Database failures should continue to propagate as errors.

13.3 Alternative Using `query()`

The same lookup can be implemented without exception-based no-result handling:

```
public Optional<Student> findById(Long id) {

    String sql = """
            SELECT id, name, email, age
            FROM students
            WHERE id = ?
```

```
                """;

        List<Student> students =
                jdbcTemplate.query(
                        sql,
                        studentRowMapper,
                        id
                );

        return students.stream().findFirst();
    }
```

`query()` returns an empty list when no student exists, which is converted into `Optional.empty()`.

The trade-off is semantic clarity:

```
queryForObject()
```

clearly communicates that exactly one row is expected.

For primary-key lookups, either approach can be used when its behaviour is understood.

---

14. Update Operation

```
public boolean update(
        Long id,
        Student student
) {

    String sql = """
            UPDATE students
            SET name = ?,
                email = ?,
                age = ?
```

```
            WHERE id = ?
            """;

    int rowsAffected = jdbcTemplate.update(
            sql,
            student.getName(),
            student.getEmail(),
            student.getAge(),
            id
    );

    return rowsAffected == 1;
}
```

Parameter order must match placeholder order:

```
First ?  → name
Second ? → email
Third ?  → age
Fourth ? → id
```

Possible affected-row counts:

```
1 → One matching student was updated
0 → No matching student was found
```

For a primary-key condition, more than one affected row should not be possible.

---

15. Delete Operation

```
public boolean deleteById(Long id) {

    String sql = """
            DELETE FROM students
            WHERE id = ?
```

```
            """;

    int rowsAffected = jdbcTemplate.update(
            sql,
            id
    );

    return rowsAffected == 1;
}
```

`jdbcTemplate.update()` is used for insert, update and delete operations because each changes database state and returns an affected-row count.

---

16. `BeanPropertyRowMapper`

A custom mapper provides explicit control:

```
private final RowMapper<Student> studentRowMapper =
        (resultSet, rowNumber) ->
                new Student(
                        resultSet.getLong("id"),
                        resultSet.getString("name"),
                        resultSet.getString("email"),
                        resultSet.getInt("age")
                );
```

Spring also provides:

```
BeanPropertyRowMapper
```

It can be configured as follows:

```
private final RowMapper<Student> studentRowMapper =
        new BeanPropertyRowMapper<>(Student.class);
```

The query remains unchanged:

```
public List<Student> findAll() {

    String sql = """
            SELECT id, name, email, age
            FROM students
            """;

    return jdbcTemplate.query(
            sql,
            studentRowMapper
    );
}
```

`BeanPropertyRowMapper` attempts to match database columns with JavaBean properties.

```
Database column → Java property

id              → id
name            → name
email           → email
age             → age
```

It can also map underscore-separated column names to camel-case properties:

```
first_name     → firstName
date_of_birth  → dateOfBirth
```

The target class generally needs:

- A no-argument constructor
- Public setter methods

Advantages

- Reduces mapper code
- Convenient for simple tables
- Useful for prototypes and straightforward mappings

Limitations

- Depends on naming conventions
- Provides less control over conversions
- Mapping problems may be less obvious
- Not ideal for complex joins
- Uses reflection and metadata-based mapping

A custom `RowMapper` is usually better when mapping logic is complex or explicit control is important.

---

17. Spring JDBC Exception Translation

17.1 Problems with `SQLException`

Raw JDBC operations throw:

```
SQLException
```

`SQLException` is a checked exception.

The compiler therefore requires either:

```
try {
    // JDBC operation
}
catch (SQLException exception) {
    // handle exception
}
```

or:

```
throws SQLException
```

`SQLException` is also very broad. The same exception type may represent:

- Invalid SQL
- Duplicate key
- Constraint violation
- Connection failure
- Authentication failure
- Deadlock
- Timeout
- Invalid column name
- SQL syntax error

Developers may need to inspect:

```
exception.getSQLState();
exception.getErrorCode();
exception.getMessage();
```

Vendor-specific error codes may differ between MySQL, PostgreSQL, Oracle and other databases.

17.2 `DataAccessException`

Spring catches `SQLException` and translates it into Spring's data-access exception hierarchy.

The root exception is:

```
DataAccessException
```

The flow is:

```
Database error
    ↓
JDBC driver throws SQLException
    ↓
JdbcTemplate catches SQLException
    ↓
Spring examines the exception
    ↓
A suitable DataAccessException subtype is thrown
```

This exception hierarchy is not tied to one particular database vendor.

17.3 Checked vs Unchecked Exceptions

`SQLException` is checked:

```
public class SQLException extends Exception
```

`DataAccessException` is unchecked:

```
public abstract class DataAccessException
        extends NestedRuntimeException
```

Repository methods therefore do not need to catch or declare every database exception.

```
public boolean save(Student student) {
    return jdbcTemplate.update(...) == 1;
}
```

When the operation fails, the relevant `DataAccessException` subtype propagates.

Many database failures cannot be meaningfully resolved inside the repository itself, for example:

- Database server is unavailable

- SQL is invalid
- Credentials are incorrect
- Required table does not exist

Such failures can propagate to centralized exception-handling code.

---

18. Common Spring Database Exceptions

18.1 `DuplicateKeyException`

Thrown when an insert or update violates a unique or primary-key constraint.

Suppose `email` is unique:

```
email VARCHAR(150) UNIQUE
```

Inserting the same email again may produce:

```
DuplicateKeyException
```

Application-specific handling may convert it into a domain exception:

```
try {
    studentRepository.save(student);
}
catch (DuplicateKeyException exception) {
    throw new EmailAlreadyExistsException(
            student.getEmail()
    );
}
```

18.2 `DataIntegrityViolationException`

Represents a broader data-integrity or constraint violation.

Examples include:

- Foreign-key violation
- Not-null violation
- Check-constraint violation
- Certain unique-constraint violations

`DuplicateKeyException` represents a more specific category of integrity failure.

18.3 `BadSqlGrammarException`

May be thrown when SQL is invalid.

Examples:

```
SELEKT id FROM students;
```

```
SELECT unknown_column FROM students;
```

18.4 `CannotGetJdbcConnectionException`

Represents a failure to obtain a database connection.

Possible causes include:

- Database server is down
- Incorrect hostname
- Incorrect port
- Network failure
- Invalid credentials
- Pool cannot provide a connection within the timeout

18.5 `EmptyResultDataAccessException`

Thrown when an operation expects at least one result but receives none.

A common example is:

```
queryForObject()
```

when no row matches the query.

18.6 `IncorrectResultSizeDataAccessException`

Thrown when the number of returned rows does not match the expected result size.

Example:

```
Expected exactly one row
Received three rows
```

---

19. Why Exception Translation Improves Portability

Suppose an application moves from MySQL to PostgreSQL.

With raw JDBC, exception-handling logic may depend on MySQL-specific error codes.

With Spring JDBC, application code can handle a common abstraction such as:

```
DuplicateKeyException
```

Spring translates the database-specific `SQLException` into a consistent exception hierarchy.

This reduces the amount of vendor-specific exception-handling code in the application.

---

20. Repository Abstraction

The controller and service layers do not need to know whether the repository uses:

- Raw JDBC
- Spring JDBC
- JPA
- Hibernate

That implementation detail remains inside the repository layer.

```
Controller
    ↓
Service
    ↓
Repository interface or repository class
    ↓
Spring JDBC / JPA / another persistence mechanism
```

This separation makes the application easier to maintain and allows persistence details to change with limited impact on higher layers.

---

21. Raw JDBC vs Spring JDBC

Area	Raw JDBC	Spring JDBC
SQL control	Full control	Full control
Connection handling	Manual	Managed through `DataSource`
Statement creation	Manual	Managed by `JdbcTemplate`
Parameter binding	Manual setter calls	Positional method arguments
Result-set iteration	Manual	Managed by `JdbcTemplate`
Row mapping	Manual	Provided through `RowMapper`
Resource cleanup	Manual	Automatic

Area	Raw JDBC	Spring JDBC
Exception model	Checked `SQLException`	Unchecked `DataAccessException` hierarchy
Connection pooling	Must be configured separately	Commonly provided through Boot and HikariCP
Boilerplate	High	Significantly reduced

22. Complete Execution Flow

For an insert operation:

```
StudentRepository.save(student)
        ↓
JdbcTemplate.update(sql, parameters)
        ↓
JdbcTemplate asks DataSource for a connection
        ↓
HikariCP provides a pooled connection
        ↓
JdbcTemplate creates PreparedStatement
        ↓
JdbcTemplate binds parameter values
        ↓
PreparedStatement executes SQL
        ↓
Database returns affected-row count
        ↓
JdbcTemplate closes the statement
        ↓
Connection is returned to the pool
        ↓
Repository receives the affected-row count
```

For a select operation:

```
StudentRepository.findAll()
        ↓
JdbcTemplate.query(sql, rowMapper)
        ↓
Connection obtained from DataSource
        ↓
PreparedStatement created and executed
```

```
        ↓
Database returns ResultSet
        ↓
JdbcTemplate iterates through each row
        ↓
RowMapper converts each row into Student
        ↓
Mapped students are collected into a List
        ↓
ResultSet and statement are closed
        ↓
Connection is returned to the pool
        ↓
List<Student> is returned
```

---

23. Key Takeaways

1. Spring JDBC still uses JDBC internally.
2. It does not remove SQL from the application.
3. `DataSource` is an abstraction that provides database connections.
4. A `DataSource` implementation may or may not use connection pooling.
5. HikariCP is the connection pool commonly used by Spring Boot.
6. `JdbcTemplate` manages the repetitive JDBC workflow.
7. `update()` is used for insert, update and delete operations.
8. `query()` is suitable when multiple rows may be returned.
9. `queryForObject()` expects exactly one result.
10. `RowMapper` converts one database row into one Java object.
11. `BeanPropertyRowMapper` reduces mapping code but provides less explicit control.
12. Spring translates `SQLException` into the `DataAccessException` hierarchy.
13. Database-specific failures should not be confused with a valid “record not found” result.
14. Spring JDBC reduces boilerplate while preserving direct control over SQL.